

BRD: MK Internal Timekeeping and Compliance App

Prepared for: Steve, Director of Growth; Mary Kennedy (Law Offices of Mary Kennedy)

Prepared by: Ganesh

Status: v1 — 14 Aug 2026, reflects the app as actually deployed and in daily use

Companion document: [PRD.md](#) — the technical/product specification this BRD sits above. Where the PRD describes *how* something works, this document describes *why* it exists and what business outcome it's accountable for.

1. Executive summary

The Law Offices of Mary Kennedy tracks time, leave, and compliance for its ~45-person immigration-law support staff (originally entirely India-based offshore staff; the team has since grown to include US-based staff too) using three manually maintained spreadsheets. Those spreadsheets are failing under their own weight — one employee's task log alone had grown to a 20 MB file that decompresses to a 700 MB XML grid — and produce no automated compliance signal beyond a single strike-count formula that itself depends on HR manually cross-referencing all three files by hand every day.

This project replaces those three spreadsheets with a single internal web application: employees log time once, and leave records, hours variance, and compliance status all derive from that one entry automatically. It was scoped and approved as an **interim tool** — built quickly, cheaply, and for easy data export — to run only until a third-party HR software pilot concludes, not as a permanent platform investment.

What was approved as a proof of concept has since become the team's actual day-to-day system: it is deployed on real production infrastructure (Google Cloud Platform), used daily by real staff with real login credentials, and has grown, through direct manager and employee feedback, to cover break tracking, a Punch In/Out clock with automatic overtime and compensation accounting, over-time pre-approval, a bug/feature ticketing system, employee self-service leave requests, and three dedicated reporting views — none of which were in the original approved scope. This document records the business case for both the original build and everything approved incrementally since.

2. Business background

2.1 The problem, in the business's own terms

HR/admin (Norine's function, with Steve and Mary also touching the data) runs time tracking across three separate spreadsheet artifacts:

Artifact	Purpose	What's actually wrong with it
Task Summary (one file per employee)	Every task an employee does, logged to the minute.	Files grow without bound and are individually degrading — one employee's file is 20 MB and decompresses to a 700 MB XML grid with over 16,000 empty repeated columns per row. Slow to open, effectively impossible to analyze in bulk.
Leave Tracker (one workbook, 57 tabs — one per person)	Leave taken, by category, plus free-text notes on make-up/compensated hours.	No computed running balance of over/under hours anywhere in the file. Compensation offsets exist only as free-text notes ("compensated 3-5 July"), so nothing reconciles automatically and a disciplinary conversation requires manually re-deriving the history.
Compliance sheet (one workbook, one tab per month)	The official record of who was compliant on which day, and the monthly strike count that can lead to pay consequences.	Every daily mark is entered by hand, after HR has already cross-referenced the other two files. The strike-count formula is the <i>only</i> automation in the entire process — everything feeding it is manual.

The real cost to the business isn't any one of these files individually — it's that **HR has to manually reconcile all three, every day, to produce one number (the strike count) that determines pay-related consequences**, using files that are simultaneously error-prone (hand-entered) and physically failing (the 700 MB file problem). That reconciliation labor, and the risk of an error in it directly affecting someone's pay, is the business problem this project solves.

2.2 Why this matters now

- The compliance sheet's own strike-count formula already implies a policy — X strikes in a month leads to a violation and, ultimately, pay consequences — but that policy was never actually documented anywhere in the files themselves before this project. The business was running a consequential process on tribal knowledge.
- The team has grown geographically (originally India-based offshore staff only; now also includes US-based staff), which the spreadsheet-based process was never built to handle — there's no clean way to represent a per-region holiday calendar, differing schedules, or cur-

rency/compliance nuance in a spreadsheet tab structure built for one office.

- A third-party HR software pilot is already planned. This project is explicitly **not** competing with that pilot — it's a bridge, built to be cheap to build, cheap to retire, and easy to export out of once the real platform is chosen.

3. Business objectives

1. **Eliminate the daily manual cross-referencing HR does across three files.** Success looks like: a compliance dashboard that shows every person's status for yesterday with zero data entry.
2. **Remove the spreadsheet-degradation risk entirely** by making the app, not the files, the system of record for time, leave, and compliance.
3. **Make the strike/violation process auditable and defensible.** Every status is computed from logged data, never hand-typed; every admin override requires a reason and is logged. This matters because strikes can lead to pay consequences — the business needs to be able to show its work.
4. **Give employees visibility they didn't have before.** Under the spreadsheet process, employees learned about a compliance problem only after the fact, from HR. The app shows an employee their own running hours balance, leave balance, and strike count in real time.
5. **Support the business's actual current shape** — a team that now spans India and the US — rather than the single-region assumption baked into the legacy files.
6. **Stay cheap and reversible.** This is explicitly interim. The build must not create a dependency the business regrets once the third-party HR pilot concludes — hence the “designed for export, not permanence” principle carried through both this document and the PRD.

4. Stakeholders

Stakeholder	Role in this project
Steve (Director of Growth)	Business sponsor; the PRD was prepared for him. Also a Super Admin user of the live system.
Mary Kennedy (Law Offices of Mary Kennedy) Norine	Firm principal; the app is built and branded for her firm. Also a Super Admin user. HR/admin — owns the manual cross-referencing process this project eliminates. Also a Super Admin user, bootstrapped as one of the first three production accounts.
Deepthi	Admin stakeholder; one of the three bootstrapped Super Admin accounts on first production deploy.
Ganesh	Scoped, built, deployed, and continues to extend the application based on direct manager/employee feedback.

Stakeholder	Role in this project
~45 employees	End users of the Employee zone (Today, My Month, Leave, Overtime, Holidays, Support, Profile) — originally all India-based offshore staff, now also including US-based staff.
Department-scoped Admins / Team Leads	A newer stakeholder group, added after the original two-role model: managers who need visibility and approval authority over their own department/direct reports, without full org-wide access.
Two Developers (currently Ganesh and Mohan)	Own the Ticketing System's triage/status queue — a fourth, independent access flag layered on top of the employee/admin role model.

5. Scope

5.1 In scope (business terms)

- Replacing all three legacy spreadsheets as the system of record for time, leave, and compliance for the duration of the interim period.
- A single place an employee logs time, requests leave, and sees their own status — once, not once per spreadsheet.
- Automatic, computed compliance status and strike counting, reproducing the legacy sheet's own verified rule exactly (so nobody's historical record changes as a side effect of switching systems).
- Role-based access matching how the business actually organizes people: individual employees, department/team-level oversight, and firm-wide administration, plus a narrow technical-support role for the ticketing system.
- Export of any admin view to Excel — the deliberate bridge to whatever the third-party HR pilot ultimately becomes.
- Everything the business has asked for incrementally since the original build (Section 7), on the same “ship it, get feedback, adjust” cadence the original nine-screen scope was delivered under.

5.2 Out of scope

- Payroll integration or actually calculating/applying pay changes. The app flags a violation; a human decides what happens next. This was a deliberate boundary from the start and remains one.
- Full historical migration of every legacy task-detail row. Roster, leave, and strike history migrate; task-level history migrates on a best-effort basis only (see PRD §2, §11).
- Anomaly detection (unusual work patterns, meta-work ratios, unauthorized project time). Recognized as valuable, deliberately deferred — the structured data this app now produces makes that analysis straightforward to build later, which is itself part of the business case for having built this at all.

- A dedicated native mobile app. The web app is responsive and reachable from a phone browser; a separate app was never justified for a ~45-person internal tool.
- Automated notifications (end-of-day reminders, strike alerts). Recognized as useful, deliberately deferred as a fast-follow rather than core scope.

5.3 Scope changes since the original approval

The original approval covered nine screens (Today, My Month for employees; Dashboard, Person Detail, Roster, Lists, Leave, Config, Export for admins). Every feature beyond that list — three-tier admin roles, break/Punch In/Out tracking, overtime pre-approval, the ticketing system, self-service leave requests, personal/bank detail self-service, bulk admin tooling, and three reporting pages — was approved incrementally, driven by direct manager and employee feedback during actual use rather than a formal change-request process. See PRD §13 for the full technical list and this document's Section 7 for the business justification behind the largest of them.

6. Business requirements

High-level business requirements, each traceable to the PRD section that specifies how it's implemented.

#	Business requirement	PRD reference
BR-1	An employee can log a full day's work in one place, with the total computed automatically — never hand-added.	PRD §4
BR-2	Leave taken, and the running hours balance (extra/short), must be computed automatically, not hand-typed into a spreadsheet cell.	PRD §5
BR-3	Compliance status and monthly strike count must be computed automatically from the same logged data, using the business's own already-verified rule (count of Missing + Partial days).	PRD §6
BR-4	Any change to a computed status must require a reason and be permanently logged, because it can affect someone's pay.	PRD §6

#	Business requirement	PRD reference
BR-5	HR/admin must be able to see every employee's status for any past day without manually cross-referencing anything.	PRD §7 (Compliance dashboard)
BR-6	Every admin view must be exportable to Excel, so the business is never locked into this tool and can hand data to the eventual third-party platform.	PRD §7, §9
BR-7	Access must reflect how the business is actually organized: individual staff, department-level managers, and firm-wide administrators — not one flat “admin” role for everyone with elevated access.	PRD §3, §13
BR-8	Employees must be able to see their own compliance standing (strikes, hours balance, leave balance) in real time, not find out after the fact from HR.	PRD §7 (My Month)
BR-9	The system must correctly support a team that spans multiple countries/regions (working-day schedules, holiday calendars), not assume a single office location.	PRD §10 open question 9, §13
BR-10	Time actually worked beyond the daily target (overtime) must be trackable and pre-approvable, separate from ordinary compliance, since it has separate payroll implications.	PRD §13
BR-11	Employees and admins need a channel to report bugs and request enhancements to the tool itself, so it keeps improving without a separate outside process.	PRD §13 (Ticketing System)

#	Business requirement	PRD reference
BR-12	Sensitive personal/financial information an employee provides (bank details, national ID numbers) must be protected from casual exposure, including from admins who can otherwise see everything.	PRD §13 (Self-service profile — numbers masked to last-4 everywhere, always)
BR-13	Switching from spreadsheets to the app must not silently change anyone's historical compliance record.	PRD §9 acceptance test — 168/168 person-months of legacy strike counts reproduced exactly

7. Business case for the largest post-approval additions

Recorded here because each represents a real, standalone business decision, not just a technical add-on:

- **Break tracking + Punch In/Out with automatic compensation.** HR previously had no live picture of who was actually clocked in versus just scheduled, and shortfall/overtime reconciliation was entirely the manual free-text process described in Section 2.1. The Punch Clock gives a live, automatic version of exactly that reconciliation, without requiring a second manual process alongside the task log.
- **Three-tier admin roles.** The original single “Admin = sees and does everything” role didn’t scale once department-level managers needed enough visibility to do their job (approve their own team’s leave/overtime, see their own team’s reports) without also being able to see or touch the entire firm’s roster, pay-sensitive overrides, and audit history. This is a business access-control decision, not just a technical one.
- **Overtime pre-approval.** Distinct from ordinary compliance tracking — the business needed a record of overtime that was *approved in advance* versus overtime that just happened, because those two have different payroll treatment.
- **Employee self-service leave requests.** The original design had admin entering all leave, matching the spreadsheet-era process. In practice this created a bottleneck and a translation step (employee tells admin, admin types it in) that added no value; letting employees submit their own request for approval removes that step while keeping admin approval authority intact.
- **The Ticketing System.** As the app took on more of the business’s actual day-to-day process, the business needed its own internal channel for reporting problems and requesting changes to the tool — rather than that traffic going through informal channels with no record or triage.
- **Holiday Management, and the decision to revert it to one shared calendar.** When the team grew to include US-based staff, the business’s first instinct was that holidays should be region-specific (a US employee shouldn’t be marked non-compliant on a US holiday if the calendar only reflected India’s). That was built and shipped. Within days, real usage showed the team actually wanted one common, shared holiday calendar for everyone regardless of location — a good example of this project’s low cost of iteration (see Section 8) working as intended: a business decision was tried, observed, and reversed in under a week with no

data loss and no schema risk, something that would have been far more expensive to do in the legacy spreadsheet process.

8. Assumptions and constraints

- **This remains an interim tool.** Nothing in this project should be read as a decision to build a permanent, standalone HR platform in-house. The moment the third-party HR software pilot concludes and is adopted, this app's job is to hand off its data cleanly (Excel export, BR-6) and step aside.
- **Real production data now exists.** What began as a proof of concept is now running against real employee data on persistent infrastructure. This changes the engineering constraints (schema changes must be additive-only; see PRD §9/§12 criterion 6) but does not change the business's ability to retire the tool whenever it chooses.
- **Low cost of iteration is a deliberate, load-bearing property,** not an accident. Because features ship incrementally and are built to be reversible (see the Holiday Management example in Section 7), the business can request a change, see it in production within days, and reverse course cheaply if it doesn't work as hoped — a materially different risk profile than a traditional big-batch software project.
- **No payroll or pay-calculation authority.** The app computes and flags; every consequence that touches actual pay remains a human decision made outside the tool, on every feature added since the original scope, not just the ones explicitly named in Section 5.2.
- **Region-of-record for staff is India and the US** as of this document's writing; the data model (per-employee `location`) accommodates this without assuming it won't grow further.

9. Risks

Risk	Mitigation
Switching systems silently changes someone's historical compliance record, undermining trust in the new system.	The acceptance test (PRD §12.3) reproduces all 168 person-months of legacy strike counts exactly before the app is trusted for anything new; this gate is re-checked on every change to the underlying computation logic.
A single vendor/hosting dependency becomes a new lock-in the business didn't intend when it approved an "interim" tool.	Hosting and auth were both built with a documented, tested migration path to an alternative (an equivalent Azure deployment script is kept in the repo alongside the live GCP one; Entra ID auth is a designed-for swap point even though self-signup password auth is what's live). Data export (BR-6) is the ultimate exit path regardless of hosting.
Sensitive employee data (bank details, national IDs) is exposed more broadly than intended now that it's digitized instead of living in scattered notes/paper.	Every sensitive number is masked to last-4 everywhere it's displayed, including to admins, the instant it's saved (BR-12) — nobody, including a Super Admin, sees the full number in the UI.

Risk	Mitigation
Feature growth beyond the original nine screens turns an “interim tool” into an unplanned permanent platform commitment by accident.	Every addition since the original approval has gone through the same lightweight, manager-driven feedback loop (Section 5.3) rather than silently scope-creeping — this document exists specifically to keep that growth visible and traceable, not to bless open-ended expansion.
A department-scoped Admin or a plain Employee sees or acts on data outside their intended authority (e.g., another department’s leave requests, another employee’s bank details).	Every scoping rule is enforced server-side on the route itself, not just hidden in the UI — verified by dedicated automated tests (<code>test_auth.py</code> ’s department-scoping tests, and equivalent tests for every other access boundary).

10. Success metrics

Restated in business terms from PRD §12 (which has the full technical detail, including what’s been met so far):

1. Employees spend less time logging a day’s work than the spreadsheet process took them, with zero manual arithmetic.
2. HR’s daily manual cross-referencing task is eliminated — the compliance dashboard shows everyone’s status for yesterday with no data entry.
3. Nobody’s historical compliance record changed as a side effect of the switch (168/168 reproduced exactly).
4. Every admin view can be handed to the eventual third-party platform as a clean Excel export.
5. The system runs on real, persistent infrastructure with real authentication — not a local demo — proving it’s actually usable as the team’s daily driver for as long as the interim period lasts.
6. Feature requests from managers/employees can be evaluated, built, and — if they don’t work out — reversed, within days, without risking the data already in the system.

11. Rollout history (business timeline)

When	What
Original build	Nine-screen POC scoped, built, and verified against the three legacy spreadsheets; 168/168 acceptance test passing.
Following weeks	Break tracking, Punch In/Out, three-tier admin roles, overtime pre-approval, the Ticketing System (built, held behind a feature flag pending its own rollout), self-service leave requests, personal/bank detail self-service, bulk admin tooling, and three reporting pages all added incrementally.

When	What
2026-07-28	Rollout plan finalized: self-signup password authentication (not waiting on Entra ID), hosted on the team's own cloud infrastructure rather than the org's existing tenant.
2026-08-01	Live production deployment on Google Cloud Platform (Cloud Run + Cloud SQL for PostgreSQL), with the first three Super Admin accounts (Deepthi, Steve, Norine) bootstrapped.
2026-08-12	Holiday Management shipped with per-country (US/India) scoping, in response to the team's geographic growth.
2026-08-14	Holiday Management reverted to a single shared calendar for everyone, after real usage showed that's what the team actually wanted — see Section 7.

12. Approval

Name	Role	Sign-off
Steve	Director of Growth, business sponsor	
Mary Kennedy	Firm principal	
Norine	HR/admin process owner	

This BRD is a living record, updated alongside the PRD as the business approves further changes — not a one-time sign-off document for a project that's already shipped and is already in daily use.